

Graph Algorithms

Complete Unit Notes

Introduction to Graphs · BFS & DFS · Topological Sort · Dijkstra · Bellman-Ford · Floyd-Warshall

Subject	Data Structures & Algorithms
Topics	Graph Theory, Shortest Path, All-Pairs
Algorithms	4 Major Algorithms with Full Worked Examples
Complexity	Time & Space Analysis for Every Algorithm

Table of Contents

1.	Introduction to Graphs	p.3
2.	Graph Representations	p.3
3.	Graph Traversals — BFS & DFS	p.4
4.	Topological Sort	p.5
5.	Single Source Shortest Path — Introduction	p.6
6.	Dijkstra's Algorithm	p.7
7.	Bellman-Ford Algorithm	p.9
8.	Comparison: Dijkstra vs Bellman-Ford	p.11
9.	All-Pairs Shortest Paths — Introduction	p.11
10.	Floyd-Warshall Algorithm	p.12
11.	Algorithm Complexity Quick Reference	p.14

1. Introduction to Graphs

What is a Graph?

A **graph** $G = (V, E)$ is a mathematical structure consisting of a set of **vertices** (nodes) V and a set of **edges** (links) E connecting pairs of vertices. Graphs model networks, maps, social connections, dependencies, and many other real-world structures.

Key Terminology

Term	Definition
Vertex (Node)	A fundamental unit. Represents an entity (city, person, task).
Edge	A connection between two vertices. Can be weighted or unweighted.
Directed Graph (Digraph)	Edges have direction: $(u,v) \neq (v,u)$. E.g., one-way streets.
Undirected Graph	Edges are bidirectional: $(u,v) = (v,u)$. E.g., friendships.
Weighted Graph	Each edge carries a numerical weight (cost, distance, time).
Adjacent	Vertex v is adjacent to u if edge (u,v) exists.
Degree	Number of edges incident to a vertex. In-degree / out-degree for digraphs.
Path	A sequence of vertices where each consecutive pair is connected by an edge.
Cycle	A path that starts and ends at the same vertex.
DAG	Directed Acyclic Graph — directed graph with no cycles.
Connected Graph	Every vertex is reachable from every other vertex.
Sparse Graph	$ E \ll V ^2$ (few edges compared to maximum possible).
Dense Graph	$ E \approx V ^2$ (many edges).

Graph Types at a Glance

Type	Directed?	Weighted?	Cycles?	Example Use Case
Simple Graph	No	No	Yes	Social network friends
Directed Graph	Yes	No	Yes	Web page hyperlinks
Weighted Graph	Either	Yes	Yes	Road maps with distances
DAG	Yes	Either	No	Task scheduling, compilation
Tree	Either	Either	No	File system, parse tree
Complete Graph K_n	No	No	Yes	All n vertices connected to all others

2. Graph Representations

A graph can be stored in memory using two primary data structures. The choice impacts time and space complexity of all algorithms.

Adjacency Matrix

A 2D array M of size $|V| \times |V|$. $M[i][j]$ = weight of edge (i,j) , or 0 / INF if no edge exists.

Example graph: $V = \{0,1,2,3\}$

Edges (directed, weighted): $0 \rightarrow 1(4)$, $0 \rightarrow 2(2)$, $1 \rightarrow 3(5)$, $2 \rightarrow 1(1)$, $2 \rightarrow 3(8)$, $3 \rightarrow 1(0)$

Adjacency Matrix:

```

      0    1    2    3
0 [ 0    4    2   INF ]
1 [ INF  0   INF  5   ]
2 [ INF  1    0    8   ]
3 [ INF  0   INF  0   ]

```

Space: $O(V^2)$

Edge lookup: $O(1)$ | Iterate neighbours: $O(V)$

Adjacency List

An array of $|V|$ lists. $List[u]$ contains all vertices v such that edge (u,v) exists, along with its weight.

Adjacency List for same graph:

```

0: [(1,4), (2,2)]
1: [(3,5)]
2: [(1,1), (3,8)]
3: [(1,0)]

```

Space: $O(V + E)$ (better for sparse graphs)

Edge lookup: $O(\text{degree}(u))$ | Iterate neighbours: $O(\text{degree}(u))$

TIP

RULE OF THUMB: Use Adjacency Matrix for dense graphs ($|E|$ close to $|V|^2$) or when $O(1)$ edge lookup is critical. Use Adjacency List for sparse graphs (most real-world graphs) and algorithms like BFS, DFS, Dijkstra that iterate neighbours.

Property	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Add edge	$O(1)$	$O(1)$
Remove edge	$O(1)$	$O(\text{degree})$
Check edge	$O(1)$	$O(\text{degree})$
Iterate neighbours	$O(V)$	$O(\text{degree})$
Best for	Dense graphs	Sparse graphs

3. Graph Traversals — BFS & DFS

Graph traversal algorithms visit every vertex and edge systematically. They form the foundation for Topological Sort, shortest path detection, cycle detection, and many other algorithms.

Breadth-First Search (BFS)

Explores all vertices at the current depth before moving deeper. Uses a **Queue (FIFO)**. Guarantees shortest path in *unweighted* graphs.

```
BFS(G, source s):
  INITIALISE all vertices: colour=WHITE, d=INF, parent=NIL
  colour[s] = GREY; d[s] = 0; parent[s] = NIL
  Q = empty queue
  ENQUEUE(Q, s)
  while Q is not empty:
    u = DEQUEUE(Q)
    for each neighbour v of u:
      if colour[v] == WHITE:           // not yet visited
        colour[v] = GREY
        d[v] = d[u] + 1
        parent[v] = u
        ENQUEUE(Q, v)
    colour[u] = BLACK                  // fully processed

Colours: WHITE=undiscovered GREY=discovered BLACK=finished
Time: O(V + E)   Space: O(V)
```

Depth-First Search (DFS)

Explores as far as possible along each branch before backtracking. Uses a **Stack** (or recursion). Records **discovery time** $d[u]$ and **finish time** $f[u]$ — crucial for Topological Sort.

```
DFS(G):
  INITIALISE all vertices: colour=WHITE, parent=NIL
  time = 0
  for each vertex u in V:
    if colour[u] == WHITE:
      DFS-VISIT(G, u)

DFS-VISIT(G, u):
  colour[u] = GREY; time = time + 1; d[u] = time
  for each neighbour v of u:
    if colour[v] == WHITE:
      parent[v] = u
      DFS-VISIT(G, v)
  colour[u] = BLACK; time = time + 1; f[u] = time

Time: O(V + E)   Space: O(V)   (recursion stack depth = O(V))
```

DFS Edge Classification

Edge Type	Condition	Significance
Tree Edge	v was WHITE when (u,v) explored	Forms the DFS forest
Back Edge	v is GREY when (u,v) explored	Indicates a cycle (in directed or undirected graphs)
Forward Edge	v is BLACK, $d[u] < d[v]$	Directed graph only; to a descendant
Cross Edge	v is BLACK, $d[u] > d[v]$	Directed graph only; between subtrees

4. Topological Sort

Definition & Concept

A **topological ordering** of a **Directed Acyclic Graph (DAG)** is a linear ordering of all vertices such that for every directed edge (u, v) , vertex u appears before vertex v in the ordering.

Topological sort is only possible on a DAG. If the graph contains a cycle, no valid topological ordering exists.

KEY RULE

IMPORTANT: Topological sort is defined ONLY for Directed Acyclic Graphs (DAGs).

A graph has a valid topological ordering if and only if it has no directed cycles.

Multiple valid orderings may exist for a single DAG.

Applications

Domain	Application
Build Systems	Determine compilation order (compile dependencies before dependents)
Task Scheduling	Order tasks where some must complete before others begin
Course Prerequisites	Find valid course enrollment order
Spreadsheets	Determine cell evaluation order
Package Managers	Install dependencies before the dependent package (npm, pip)
Circuit Design	Logic gate evaluation order in combinational circuits

Method 1: DFS-Based Topological Sort (CLRS)

Run DFS. As each vertex finishes (colour turns BLACK), prepend it to a linked list. The resulting list is a topological ordering.

```
TOPOLOGICAL-SORT(G):
    result_list = empty linked list
    Run DFS on G
    As each vertex u is finished (f[u] set), INSERT u at FRONT of result_list
    return result_list
```

Key insight: vertex with the HIGHEST finish time appears FIRST.

A back edge in DFS means a cycle exists -> topological sort not possible.

Time Complexity: $O(V + E)$ (same as DFS)

Space Complexity: $O(V)$ (for the result list + DFS stack)

Method 2: Kahn's Algorithm (In-Degree / BFS-Based)

Repeatedly remove vertices with in-degree 0 (no dependencies). Uses a queue. Detects cycles if not all vertices are processed.

```

KAHN-TOPOLOGICAL-SORT(G):
  Compute in-degree[v] for all v in V
  Q = queue of all vertices with in-degree == 0
  result = []
  count = 0
  while Q is not empty:
    u = DEQUEUE(Q)
    APPEND u to result
    count = count + 1
    for each neighbour v of u:
      in-degree[v] = in-degree[v] - 1
      if in-degree[v] == 0:
        ENQUEUE(Q, v)
  if count != |V|:
    PRINT 'Graph has a cycle – topological sort impossible'
  return result

Time Complexity: O(V + E)
Space Complexity: O(V)

```

Worked Example — Course Prerequisites

Courses: A, B, C, D, E, F. Prerequisites (edges): A→C, A→D, B→D, B→E, C→F, D→F, E→F

Graph (DAG):

```

      A ---> C ---> F
      |       ^   ^
      +-> D ---+   |
      B ---> D     |
      |           |
      +----> E -----+

```

In-degrees: A=0 B=0 C=1 D=2 E=1 F=3

Kahn's Steps:

- Step 1: Q = {A, B} (in-degree 0)
- Step 2: Process A -> result=[A], reduce C(0), D(1) -> Q={B,C}
- Step 3: Process B -> result=[A,B], reduce D(0), E(0) -> Q={C,D,E}
- Step 4: Process C -> result=[A,B,C], reduce F(2) -> Q={D,E}
- Step 5: Process D -> result=[A,B,C,D], reduce F(1) -> Q={E}
- Step 6: Process E -> result=[A,B,C,D,E], reduce F(0) -> Q={F}
- Step 7: Process F -> result=[A,B,C,D,E,F]

Valid ordering: A, B, C, D, E, F (take A and B first – no prerequisites)

NOTE

Multiple valid orderings exist. E.g., B, A, D, C, E, F is also valid.
 DFS method gives one valid ordering; Kahn's can give different orderings depending on queue implementation (priority queue gives lexicographic order).

5. Single Source Shortest Paths — Introduction

Problem Statement

Given a weighted directed graph $G = (V, E, w)$ and a source vertex s , find the shortest-weight path from s to every other vertex v in V . The **shortest path weight** $d(s, v)$ is the minimum sum of edge weights over all paths from s to v .

Core Concepts

Concept	Description
Relaxation	Update $d[v] = d[u] + w(u, v)$ if $d[u] + w(u, v) < d[v]$. Central operation in all shortest path algorithms.
$d[v]$	Current estimate of shortest-path weight from s to v . Initialised to ∞ .
$\text{parent}[v]$	Predecessor of v on the shortest-path tree. Used to reconstruct the path.
Negative weight	Edge weight < 0 . Dijkstra cannot handle these. Bellman-Ford can.
Negative cycle	A cycle whose total weight is negative. Makes shortest paths undefined (keep going around = $-\infty$). Bellman-Ford detects these.

Initialisation and Relaxation (Common to All SSSP Algorithms)

```

INITIALISE-SINGLE-SOURCE( $G, s$ ):
  for each vertex  $v$  in  $G.V$ :
     $d[v] = \infty$ 
     $\text{parent}[v] = \text{NIL}$ 
   $d[s] = 0$ 

RELAX( $u, v, w$ ):
  if  $d[u] + w(u, v) < d[v]$ :
     $d[v] = d[u] + w(u, v)$ 
     $\text{parent}[v] = u$ 
  
```

Algorithm	Graph Type	Negative Weights?	Negative Cycles?	Time Complexity
Dijkstra	Directed / Undirected	NO	N/A	$O((V+E) \log V)$
Bellman-Ford	Directed	YES	Detects	$O(VE)$
DAG Shortest	DAG only	YES	None (DAG)	$O(V+E)$
Floyd-Warshall	All-pairs, Directed	YES	Detects	$O(V^3)$

6. Dijkstra's Algorithm

Overview

Dijkstra's algorithm (Edsger Dijkstra, 1956) solves the single-source shortest path problem for graphs with **non-negative edge weights**. It uses a greedy strategy: at each step, it finalises the vertex with the current minimum distance estimate.

CONSTRAINT

REQUIREMENT: All edge weights must be ≥ 0 .
 If any negative edges exist, Dijkstra's may produce incorrect results.
 Use Bellman-Ford for graphs with negative edge weights.

Core Idea — Greedy Invariant

Dijkstra maintains a set S of vertices whose shortest distance from s is finalised. At each iteration it extracts from the min-priority queue the vertex u with minimum $d[u]$, adds it to S , and relaxes all outgoing edges from u . The greedy choice is safe because all weights are non-negative.

Pseudocode

```

DIJKSTRA( $G, w, s$ ):
  INITIALISE-SINGLE-SOURCE( $G, s$ )           //  $d[s]=0, d[v]=\text{INF}$  for all others
   $S = \text{empty set}$                           // finalised vertices
   $Q = \text{MIN-PRIORITY-QUEUE}(V)$            // keyed by  $d[v]$ 

  while  $Q$  is not empty:
     $u = \text{EXTRACT-MIN}(Q)$                   // vertex with smallest  $d[u]$ 
     $S = S \cup \{u\}$ 
    for each edge  $(u, v)$  in  $G.\text{Adj}[u]$ :
      RELAX( $u, v, w$ )                      // update  $d[v]$  if shorter path found
      if  $d[v]$  was updated:                  // decrease-key in priority queue
        DECREASE-KEY( $Q, v, d[v]$ )

  Result:  $d[v] = \text{shortest distance from } s \text{ to } v \text{ for all } v$ 
          $\text{parent}[v] = \text{predecessor on shortest path}$ 

```

Time Complexity Analysis

Priority Queue Impl.	EXTRACT-MIN	DECREASE-KEY	Total
Binary Heap	$O(\log V)$	$O(\log V)$	$O((V + E) \log V)$
Fibonacci Heap	$O(\log V)^*$	$O(1)^*$	$O(V \log V + E)$ [amortised]
Simple Array	$O(V)$	$O(1)$	$O(V^2)$ — good for dense graphs

Detailed Worked Example

Graph (directed, weighted): Vertices $\{A, B, C, D, E\}$, Source = A

Edges:

A->B weight 4 A->C weight 2
 B->C weight 5 B->D weight 10
 C->E weight 3 D->E weight 4
 E->D weight 4 C->B weight 1

Initial: $d[A]=0$ $d[B]=\text{INF}$ $d[C]=\text{INF}$ $d[D]=\text{INF}$ $d[E]=\text{INF}$
 $Q = \{A(0), B(\text{INF}), C(\text{INF}), D(\text{INF}), E(\text{INF})\}$

Step	Extract-Min	Relax Edges	d[A]	d[B]	d[C]	d[D]	d[E]
1	A (d=0)	A->B: $0+4=4$ A->C: $0+2=2$	0	4	2	INF	INF
2	C (d=2)	C->E: $2+3=5$ C->B: $2+1=3$	0	3	2	INF	5
3	B (d=3)	B->C: $3+5=8 > 2$ (skip) B->D: $3+10=13$	0	3	2	13	5
4	E (d=5)	E->D: $5+4=9 < 13$ -> update D=9	0	3	2	9	5
5	D (d=9)	No outgoing edges from D	0	3	2	9	5

Final Shortest Paths from A:

A -> A : 0 (trivial)
 A -> B : 3 (path: A -> C -> B)
 A -> C : 2 (path: A -> C)
 A -> D : 9 (path: A -> C -> E -> D)
 A -> E : 5 (path: A -> C -> E)

Path Reconstruction (follow parent[] pointers backwards):

parent[B]=C parent[C]=A parent[D]=E parent[E]=C
 D's path: D <- E <- C <- A => A -> C -> E -> D

Correctness — Why Dijkstra Works

At the moment a vertex u is extracted from the min-heap, $d[u]$ is the true shortest-path distance. Proof by induction: any path that bypasses u and goes through an unvisited vertex v must have weight $\geq d[u]$ (since all weights are non-negative), so no shorter path can exist.

7. Bellman-Ford Algorithm

Overview

The Bellman-Ford algorithm (Richard Bellman, 1958; Lester Ford, 1956) solves the single-source shortest path problem for graphs that may contain **negative-weight edges**. It also detects **negative-weight cycles**. It is slower than Dijkstra but more general.

Core Idea

Relax **all $|E|$ edges exactly $|V|-1$ times**. After k iterations of relaxation, $d[v]$ holds the shortest path from s to v using at most k edges. Since the shortest simple path can use at most $|V|-1$ edges, after $|V|-1$ passes all true shortest distances are found.

Pseudocode

```
BELLMAN-FORD( $G, w, s$ ):
    INITIALISE-SINGLE-SOURCE( $G, s$ )           //  $d[s]=0, d[v]=\text{INF}$  for all others

    // Phase 1: Relax all edges  $|V|-1$  times
    for  $i = 1$  to  $|V| - 1$ :
        for each edge  $(u, v)$  in  $G.E$ :
            RELAX( $u, v, w$ )

    // Phase 2: Detect negative cycles
    for each edge  $(u, v)$  in  $G.E$ :
        if  $d[u] + w(u, v) < d[v]$ :
            return FALSE // negative-weight cycle exists

    return TRUE // no negative cycle,  $d[v]$  are correct
```

Time Complexity: $O(V * E) = O(V^3)$ for dense graphs
 Space Complexity: $O(V)$

Why $|V|-1$ Iterations?

Any shortest simple path visits at most $|V|$ vertices and thus uses at most $|V|-1$ edges. Each pass over all edges guarantees that paths using one more edge are discovered. After $|V|-1$ passes, all shortest simple paths are found. If a $(|V|)$ -th pass still reduces any $d[v]$, a negative cycle exists.

Negative Cycle Detection

NEGATIVE CYCLE

After $|V|-1$ relaxation passes, perform one more pass over all edges.
 If $d[u] + w(u,v) < d[v]$ for any edge (u,v) , then v lies on or is reachable from a negative-weight cycle. The algorithm returns FALSE in this case.
 Dijkstra cannot detect negative cycles; Bellman-Ford can.

Detailed Worked Example

Graph (directed, weighted with negative edge): $V=\{A,B,C,D,E\}$, Source=A

Edges:

```

A->B  6    A->D  7
B->C  5    B->D  8    B->E  -4
C->B  -2
D->C  -3    D->E  9
E->A  2    E->C  7

```

Initial: $d[A]=0$ $d[B]=\text{INF}$ $d[C]=\text{INF}$ $d[D]=\text{INF}$ $d[E]=\text{INF}$
 $|V|=5$ so 4 passes needed

Pas s	d[A]	d[B]	d[C]	d[D]	d[E]	Key Updates
Init	0	INF	INF	INF	INF	—
1	0	6	INF	7	2	A->B=6, A->D=7, B->E=6+(-4)=2
2	0	6	4	7	2	D->C=7+(-3)=4, B->E already 2
3	0	2	4	7	2	C->B=4+(-2)=2
4	0	2	4	7	-2	B->E=2+(-4)=-2

Final Shortest Paths from A:

```

A -> A : 0
A -> B : 2    (A->D->C->B)
A -> C : 4    (A->D->C)
A -> D : 7    (A->D)
A -> E : -2   (A->D->C->B->E)

```

Negative Cycle Check (5th pass over all edges):

No edge (u,v) satisfies $d[u]+w(u,v) < d[v]$ $\Rightarrow$ No negative cycle.
 Algorithm returns TRUE; distances are correct.

Negative Cycle Detection Example

Add edge E->B with weight -10:

After 4 passes, check edge (E->B): $d[E]+(-10) = -2-10 = -12 < d[B]=2$
 $d[B]$ would still decrease $\Rightarrow$ NEGATIVE CYCLE DETECTED
 The cycle B->E->B has total weight $(-4)+(-10) = -14 < 0$
 Algorithm returns FALSE (shortest paths are undefined / -INF).

8. Comparison: Dijkstra vs Bellman-Ford

Feature	Dijkstra	Bellman-Ford
Strategy	Greedy	Dynamic Programming
Edge weight constraint	Non-negative only (≥ 0)	Any weight (positive, zero, negative)
Negative cycle	Cannot detect / incorrect	Detects and reports
Time complexity	$O((V+E) \log V)$ with heap	$O(V * E)$
Space complexity	$O(V)$	$O(V)$
Graph type	Directed or undirected	Directed (use transformed for undirected)
When to use	Road maps, GPS, non-neg graphs	Finance (currency arbitrage), neg edges
Correctness proof	Greedy invariant + induction	Induction on path length k
Practical speed	Much faster in practice	Slower; used when negative edges exist

Shortest Path on a DAG (Bonus)

For a DAG, we can find single-source shortest paths even with negative edges in $O(V+E)$ — faster than both Dijkstra and Bellman-Ford — by relaxing edges in topological order.

```

DAG-SHORTEST-PATHS( $G, w, s$ ):
    Topologically sort the vertices of  $G$ 
    INITIALISE-SINGLE-SOURCE( $G, s$ )
    for each vertex  $u$  in topological order:
        for each vertex  $v$  in  $G.Adj[u]$ :
            RELAX( $u, v, w$ )

Time:  $O(V + E)$     Works with negative edges (no cycles in DAG)

```

9. All-Pairs Shortest Paths — Introduction

Problem Statement

Find the shortest path between **every pair of vertices** (i, j) in a graph. Result is a $|V| \times |V|$ distance matrix D where $D[i][j]$ is the shortest path weight from vertex i to vertex j.

Approaches

Method	Idea	Time
Run SSSP from each vertex	Run Dijkstra $ V $ times	$O(V(V+E) \log V)$ — good for sparse+non-neg
Run SSSP from each vertex	Run Bellman-Ford $ V $ times	$O(V^2 * E)$ — handles negative edges
Matrix Multiplication	DP on path length	$O(V^3 \log V)$
Floyd-Warshall	DP on intermediate vertices	$O(V^3)$ — simple and practical
Johnson's Algorithm	Reweighting + Dijkstra from each v	$O(V^2 \log V + VE)$ — sparse+neg edges

SUMMARY

Floyd-Warshall is the standard algorithm for All-Pairs Shortest Paths in dense graphs. It handles negative edges, detects negative cycles, and has simple $O(V^3)$ implementation. For sparse graphs with non-negative weights, running Dijkstra V times is faster.

10. Floyd-Warshall Algorithm

Overview & Intuition

Floyd-Warshall (Robert Floyd, 1962; Stephen Warshall, 1962) uses **dynamic programming** to compute all-pairs shortest paths. It considers each vertex k as a potential intermediate vertex and asks: *"Is going through vertex k shorter than the current known path?"*

Recurrence Relation

Define $d(i, j, k)$ = weight of shortest path from i to j using only vertices $\{1, 2, \dots, k\}$ as intermediate vertices.

Base case: $d(i, j, 0) = w(i, j)$ if edge (i, j) exists
 $= 0$ if $i == j$
 $= \text{INF}$ otherwise

Recurrence: $d(i, j, k) = \min(d(i, j, k-1), d(i, k, k-1) + d(k, j, k-1))$
 $\quad \quad \quad \wedge \quad \quad \quad \wedge$
 $\quad \quad \quad (\text{don't use } k) \quad \quad (\text{go through } k)$

After processing all k : $D[i][j]$ = shortest path from i to j .

Pseudocode

```

FLOYD-WARSHALL(W):
    n = number of vertices
    D = W                                // D is |V|x|V| weight matrix
    P = |V|x|V| predecessor matrix      // P[i][j] = predecessor of j on path i->j

    // Initialise predecessor matrix
    for i = 1 to n:
        for j = 1 to n:
            if i == j or W[i][j] == INF:
                P[i][j] = NIL
            else:
                P[i][j] = i

    // Main DP loop
    for k = 1 to n:                      // try each vertex as intermediate
        for i = 1 to n:
            for j = 1 to n:
                if D[i][k] + D[k][j] < D[i][j]:
                    D[i][j] = D[i][k] + D[k][j]
                    P[i][j] = P[k][j]    // path to j now goes through k

    // Negative cycle check
    for i = 1 to n:
        if D[i][i] < 0:                  // negative cycle reachable from i
            report 'Negative cycle detected'

    return D, P

Time:  $O(V^3)$     Space:  $O(V^2)$ 

```

Path Reconstruction

```

PRINT-ALL-PAIRS-SHORTEST-PATH(P, i, j):
    if i == j:
        PRINT i
    else if P[i][j] == NIL:
        PRINT 'No path from i to j exists'
    else:
        PRINT-ALL-PAIRS-SHORTEST-PATH(P, i, P[i][j])
        PRINT j

```

Detailed Worked Example

Graph: $V=\{1,2,3,4\}$, directed weighted graph with negative edge.

```

Edges (directed):  1->2 (3)   1->3 (8)   1->4 (INF)
                  2->1 (INF) 2->3 (INF) 2->4 (1)
                  3->1 (INF) 3->2 (4)   3->4 (INF)
                  4->1 (2)   4->2 (INF) 4->3 (-5)

```

Initial Weight Matrix $D(0)$ [INF shown as --]

D(0)	1	2	3	4
1	0	3	8	--
2	--	0	--	1
3	--	4	0	--
4	2	--	-5	0

After k=1 (using vertex 1 as intermediate)

Check: can going through vertex 1 improve any path?

$D[2][1] = \text{INF}$ $\rightarrow$ no $i \rightarrow 1 \rightarrow j$ improvements from vertex 2 row

$D[3][1] = \text{INF}$ $\rightarrow$ no improvements

$D[4][1] = 2$ $\rightarrow D[4][2] = \min(\text{INF}, D[4][1] + D[1][2]) = \min(\text{INF}, 2+3) = 5$

$D[4][3] = \min(-5, D[4][1] + D[1][3]) = \min(-5, 2+8) = -5$ (no change)

Result after k=1: $D[4][2]$ updated to 5

After k=2 (using vertex 2 as intermediate)

$D[1][4] = \min(\text{INF}, D[1][2] + D[2][4]) = \min(\text{INF}, 3+1) = 4$ <- NEW

$D[3][4] = \min(\text{INF}, D[3][2] + D[2][4]) = \min(\text{INF}, 4+1) = 5$ <- NEW

$D[4][4] = \min(0, D[4][2] + D[2][4]) = \min(0, 5+1) = 0$ (no change)

After k=3 (using vertex 3 as intermediate)

$D[1][2] = \min(3, D[1][3] + D[3][2]) = \min(3, 8+4) = 3$ (no change)

$D[2][3] = \min(\text{INF}, D[2][3] + D[3][3]) \rightarrow D[4][3] = -5$ helps via k=4 step

$D[4][2] = \min(5, D[4][3] + D[3][2]) = \min(5, -5+4) = -1$ <- NEW

After k=4 (using vertex 4 as intermediate)

$D[1][3] = \min(8, D[1][4] + D[4][3]) = \min(8, 4+(-5)) = -1$ <- NEW

$D[2][1] = \min(\text{INF}, D[2][4] + D[4][1]) = \min(\text{INF}, 1+2) = 3$ <- NEW

$D[2][3] = \min(\text{INF}, D[2][4] + D[4][3]) = \min(\text{INF}, 1+(-5)) = -4$ <- NEW

$D[3][1] = \min(\text{INF}, D[3][4] + D[4][1]) = \min(\text{INF}, 5+2) = 7$ <- NEW

$D[3][3] = \min(0, D[3][4] + D[4][3]) = \min(0, 5+(-5)) = 0$ (no change)

Final Distance Matrix D(4) [all-pairs shortest paths]

D(final)	1	2	3	4
1	0	3	-1	4
2	3	0	-4	1
3	7	4	0	5
4	2	-1	-5	0

Reading results:

Shortest path 1->3: weight -1 (path: 1->2->4->3 i.e. 1->2 w=3, 2->4 w=1, 4->3 w=-5)

Shortest path 2->3: weight -4 (path: 2->4->3 i.e. 2->4 w=1, 4->3 w=-5)

Shortest path 4->2: weight -1 (path: 4->3->2 i.e. 4->3 w=-5, 3->2 w=4)

Negative Cycle Check: all $D[i][i] = 0 \Rightarrow$ No negative cycle.

DETECT

Negative Cycle Detection: If any $D[i][i] < 0$ after Floyd-Warshall, vertex i is on a negative-weight cycle. All shortest paths through that cycle become undefined (-infinity).

11. Algorithm Complexity Quick Reference

All Graph Algorithms — Summary Table

Algorithm	Type	Time	Space	Neg Wt?	Neg Cycle?	Key Data Structure
BFS	Traversal	$O(V+E)$	$O(V)$	N/A	N/A	Queue (FIFO)
DFS	Traversal	$O(V+E)$	$O(V)$	N/A	Detects	Stack / Recursion
Topo Sort (DFS)	DAG	$O(V+E)$	$O(V)$	N/A	Detects	DFS + output list
Topo Sort (Kahn)	DAG	$O(V+E)$	$O(V)$	N/A	Detects	In-degree + Queue
Dijkstra (heap)	SSSP	$O((V+E)\log V)$	$O(V)$	NO	No	Min-Priority Queue
Dijkstra (array)	SSSP	$O(V^2)$	$O(V)$	NO	No	Array
Bellman-Ford	SSSP	$O(VE)$	$O(V)$	YES	Detects	Edge list
DAG Shortest	SSSP	$O(V+E)$	$O(V)$	YES	N/A	Topo order
Floyd-Warshall	APSP	$O(V^3)$	$O(V^2)$	YES	Detects	2D matrix
Johnson	APSP	$O(V^2 \log V + VE)$	$O(V^2)$	YES	Detects	Heap + reweighting

Decision Guide — Which Algorithm to Use?

Is the problem Single-Source or All-Pairs?

Single-Source (from one vertex to all others):

Are there negative edges?

NO --> Use Dijkstra ($O((V+E)\log V)$ with binary heap)

[Faster; use array implementation for dense graphs $O(V^2)$]

YES --> Use Bellman-Ford ($O(VE)$)

[Also detects negative cycles]

Graph is a DAG? --> DAG Shortest Paths ($O(V+E)$) [fastest, handles neg]

All-Pairs (between every pair of vertices):

Dense graph OR negative edges?

YES --> Use Floyd-Warshall ($O(V^3)$, simple to implement)

NO (sparse + non-neg) --> Run Dijkstra from every vertex

$O(V(V+E)\log V)$ -- beats Floyd for sparse

Sparse + negative? --> Johnson's Algorithm ($O(V^2 \log V + VE)$)

Properties of Shortest Path Problems

Property	Statement
Optimal Substructure	Subpaths of shortest paths are themselves shortest paths.
Triangle Inequality	$d(s,v) \leq d(s,u) + w(u,v)$ for all edges (u,v) .

Property	Statement
Upper Bound	$d[v] \geq$ true shortest path weight. Only decreases, never increases.
No-Path Property	If no path $s \rightarrow v$ exists, then $d[v] = \text{INF}$ forever.
Convergence	After relaxing edge (u,v) : if $d[u] = \text{true SP}$, then $d[v] = \text{true SP}$.
Relaxation Order	Dijkstra relaxes in optimal order; Bellman-Ford brute-forces all orders.